

M5Stack AtomS3R – Beschreibung

Dieses Infoblatt bietet eine kompakte Übersicht über den **M5Stack AtomS3R** Mini-Controller. Der Controller basiert auf dem leistungsstarken ESP32-S3 Chip und wird in diesem Projekt direkt über **ESPHome** mittels YAML-Konfigurationen gesteuert. Eine klassische Programmierung in C oder über die Arduino IDE ist daher nicht erforderlich, das System baut im Hintergrund jedoch darauf auf.



1. Technische Kernspezifikationen

Der AtomS3R kombiniert hohe Rechenleistung, drahtlose Kommunikation und präzise Sensorik auf minimalem Raum (ca. 24 x 24 mm).

Komponente	Spezifikation / Funktion für das Projekt
Hauptprozessor (SoC)	ESP32-S3-PICO-1-N8R8 (Dual-Core Prozessor, bis zu 240 MHz)
Speicher	8 MB Flash-Speicher & 8 MB PSRAM (großzügiger Arbeitsspeicher für Datenpufferung)
Funkverbindung	2,4 GHz Wi-Fi mit verbesserter 3D-Antenne (wichtig für die CPS-Netzwirkkommunikation)
PC-Schnittstelle	USB Type-C (für Stromversorgung, Daten-Logs und das erste Flashen)

2. Physikalische Schnittstellen & Pinbelegung (Pinout)

Um den Controller in einer physikalischen Umgebung einzusetzen, müssen externe Aktoren (z. B. Relais, LEDs, Motoren) oder zusätzliche Sensoren angeschlossen werden. Der AtomS3R bietet hierfür zwei primäre Schnittstellen:

A. Der Grove-Anschluss (HY2.0-4P / Port A)

Dieser rote, verpolungssichere Steckanschluss ist perfekt für das schnelle Zusammenspiel mit dem I2C-Bus. Viele Erweiterungssensoren lassen sich hier direkt ohne Löten per Kabel anstecken.

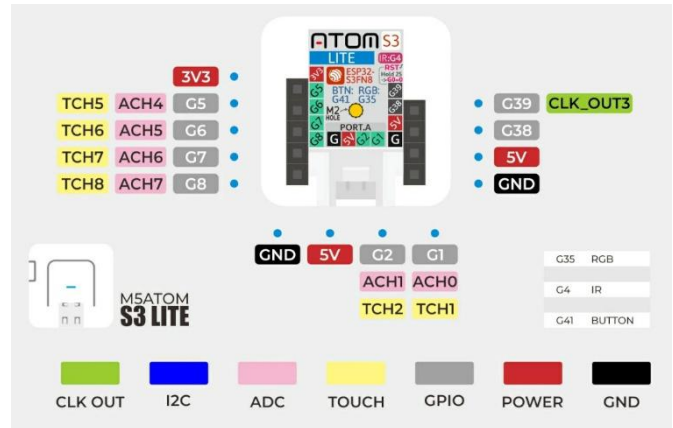
Kabelfarbe	Standardfunktion	GPIO-Pin in ESPHome
Schwarz	GND (Masse / Minuspol)	-
Rot	5V Stromversorgung (Pluspol)	-
Gelb	SDA (I2C-Datenleitung) / Digitaler I/O	GPIO2
Weiß	SCL (I2C-Taktleitung) / Digitaler I/O	GPIO1

B. Erweiterungs-Pins auf der Unterseite (Bottom Pins)

An der Unterseite des Gehäuses befinden sich freiliegende Pin-Header. Diese können auf ein Breadboard gesteckt oder für eigene Schaltungen genutzt werden:

- **Freie GPIO-Pins:** G5, G6, G7, G8, G38, G39 (Diese Pins können in ESPHome flexibel als digitaler Eingang, digitaler Ausgang, PWM-Ausgang oder analoger Eingang definiert werden).
- **Spannungs-Pins:** 5V (Stromeingang oder Ausgang), 3.3V (Ausgang für sensible Sensorik), GND.

Hinweis für die Konfiguration: In ESPHome spricht ihr die Pins einfach mit ihrer GPIO-Nummer an (z. B. pin: GPIO5).



3. Integrierte Sensorik & Komponenten

Das Besondere am AtomS3R ist, dass wichtige Komponenten für ein cyber-physikalisches System bereits fest auf der Platine verbaut sind. Ihr müsst sie nicht extra verkabeln:

- **9-Achsen-Sensorsystem (IMU):** Eine Kombination aus dem BMI270 (6-Achsen-Beschleunigung & Gyroskop) und dem BMM150 (3-Achsen-Magnetometer). Damit misst der Controller Bewegungen, Erschütterungen, Neigungswinkel und sogar die Himmelsrichtung (Kompass). Perfekt für Robotik oder Zustandsüberwachungen.
- **Infrarot-Sender (IR-LED):** Ermöglicht es dem Controller, optische Signale auszusenden, um z. B. Laborgeräte, Fernseher oder andere CPS-Module drahtlos anzusteuern.
- **Programmierbarer Haupttaster:** Das Gehäusedisplay fungiert gleichzeitig als mechanischer Taster (Knopf). Drückt man auf das Display, wird ein Signal ausgelöst (intern verdrahtet auf GPIO41), was sich ideal für Benutzerinteraktionen eignet.
- **Display (0.85" IPS LCD):** Dient zur Anzeige kleinerer Statusinformationen, steht aber für die reine Steuerungs- und Vernetzungslogik dieses Projekts im Hintergrund.

4. Steuerung über ESPHome (statt klassischem Code)

Normalerweise müsste dieser Controller in der Sprache C++ (z. B. über die Arduino IDE) programmiert werden. Um den Fokus voll auf das CPS-Projekt zu legen, nutzen wir stattdessen **ESPHome**. Ihr schreibt keinen Programmiercode, sondern beschreibt die Hardware-Struktur in einer einfachen Konfigurationsdatei (YAML-Format).

Die Vorteile auf einen Blick:

1. **Fehlervermeidung:** Keine Sorge vor vergessenen Semikolons oder komplexer Speicherverwaltung.
2. **Automatisches Wi-Fi & API:** Der Controller verbindet sich selbstständig mit dem Netzwerk und stellt Daten direkt bereit.
3. **Drahtlose Updates (OTA):** Nach dem ersten Flashen über USB-C können spätere Änderungen direkt per WLAN aufgespielt werden.

5. ESPHome Core-Strukturen & Konfigurationsblöcke

Für die CPS-Konfiguration des AtomS3R stellt ESPHome verschiedene vordefinierte YAML-Blöcke (Komponenten) zur Verfügung. Hier sind die wichtigsten Grundlagen für eure Projektdateien:

Struktur / Block	Beschreibung & Funktion im CPS-Projekt	Kurzes YAML-Beispiel
esphome & esp32	Definiert die grundlegenden Projekteigenschaften und legt das genaue Zielboard fest. Für den AtomS3R wird die ESP32-S3-Architektur deklariert.	<pre>esp32: board: esp32-s3-devkitc-1 framework: type: arduino</pre>
wifi	Konfiguriert die Funkverbindung. Ermöglicht dem Controller, sich automatisch mit dem Schul- oder Labornetzwerk zu verbinden und erreichbar zu sein.	<pre>wifi: ssid: "Schulnetzwerk" password: "Geheim123"</pre>
mqtt	Die wichtigste Schnittstelle für vernetzte Systeme (M2M). Sendet Sensordaten an einen zentralen Broker und empfängt Steuerbefehle für Aktoren.	<pre>mqtt: broker: 10.0.0.5 topic_prefix: atom_cps</pre>
i2c	Aktiviert den Datenbus für den roten Grove-Anschluss (Port A) sowie die intern verbauten Bewegungssensoren (SDA=GPIO2, SCL=GPIO1).	<pre>i2c: sda: GPIO2 scl: GPIO1 scan: true</pre>

Struktur / Block	Beschreibung & Funktion im CPS-Projekt	Kurzes YAML-Beispiel
binary_sensor	Verarbeitet digitale Zustandseingänge (AN/AUS). Wird genutzt, um z. B. den integrierten Display-Taster (GPIO41) als Schalter abzufragen.	<pre>binary_sensor: - platform: gpio pin: GPIO41 name: "Display-Taste" inverted: true</pre>
sensor	Verarbeitet kontinuierliche Messwerte (Zahlen/Analogwerte). Liest z. B. die Beschleunigung oder Temperatur der integrierten BMI270-IMU aus.	<pre>sensor: - platform: bmi270 accelerometer_x: name: "Beschleunigung X"</pre>
light	Steuert Leuchtmedien. Kann für externe adressierbare LED-Streifen (NeoPixel) an den GPIOs oder die PWM-Regelung von Signalleuchten genutzt werden.	<pre>light: - platform: status_led name: "Status LED" pin: GPIO35</pre>

Weitere YAML-Strukturen mit ESPHome

ESPHome bietet noch eine ganze Palette an mächtigen Werkzeugen, um aus dem AtomS3R ein echtes, autonomes CPS-Device zu machen. Wenn die Grundlagen erst einmal klar sind, lässt sich die YAML-Datei um folgende logische Strukturen erweitern:

1. Lokale Logik & "Edge Computing"

CPS-Systeme sollten im Idealfall auch dann noch reagieren, wenn die Netzwerkverbindung kurz abreißt. Dafür bietet ESPHome Strukturen, um Logik direkt auf dem AtomS3R auszuführen:

- **script**: Erlaubt es, eigene Befehlsabfolgen (Aktionsketten) zu definieren. Schüler können hier z. B. eine Sequenz wie „LED an, 2 Sekunden warten, LED aus“ programmieren, die dann von verschiedenen Auslösern aufgerufen werden kann.
- **interval**: Ein zeitlicher Taktgeber. Damit lässt sich festlegen, dass eine bestimmte Aktion zyklisch (z. B. alle 10 Sekunden) ausgeführt wird, ohne dass von außen ein Befehl kommen muss.
- **globals**: Globale Variablen. Hiermit können Werte (wie Zählerstände oder Schwellenwerte) direkt im Speicher des ESP32 abgelegt und blockübergreifend verändert werden.

2. Erweiterte Interaktion & Aktoren

Zusätzlich zu reinen Sensoren und Lichtern braucht man im Labor oft Schnittstellen zur gezielten Steuerung:

- **switch**: Repräsentiert klassische Schalter (AN/AUS). Perfekt, um über die unterseitigen Pins z. B. ein Relais, ein Ventil oder eine Pumpe anzusteuern.
- **button**: Im Gegensatz zum `binary_sensor` (der auf physikalische Knöpfe reagiert) ist dies ein virtueller „Taster“ für das Dashboard. Drückt man ihn im Webinterface, wird einmalig eine Aktion auf dem AtomS3R ausgelöst.
- **output**: Die Low-Level-Ansteuerung für Hardware-Pins. Besonders relevant für **PWM** (Pulsweitenmodulation), um Motorgeschwindigkeiten zu regeln oder Summer (Buzzer) in der Tonhöhe anzusteuern.

3. Spezifische Hardware-Features des AtomS3R

Der kleine Controller hat noch ein paar Spezialitäten an Bord, die über diese Blöcke aktiviert werden:

- **remote_transmitter**: Aktiviert die integrierte **Infrarot-LED** des AtomS3R. Damit können die Schüler den Controller so konfigurieren, dass er IR-Codes aussendet, um z. B. Laborgeräte oder Fernseher im Raum zu steuern.

- **display**: Auch wenn das Display im Projekt eine Nebenrolle spielt: Über diesen Block (meist in Kombination mit der SPI- oder I2C-Schnittstelle) lässt sich der winzige Bildschirm ansteuern, um z. B. die aktuelle IP-Adresse oder Fehlermeldungen direkt im CPS-Modul anzuzeigen.

4. System, Diagnose & Datenkomfort

Für den reibungslosen Laborbetrieb und die Überwachung der Geräte sind diese Strukturen extrem hilfreich:

- **ota (Over-The-Air)**: Ermöglicht kabellose Updates. Sobald die Basis-Konfiguration einmal per USB-C aufgespielt ist, können die Schüler ihren Code komplett über das WLAN aktualisieren. Das schont die USB-Buchsen im Klassenzimmer massiv.
- **logger**: Das Diagnose-Werkzeug. Es sendet Textmeldungen (z. B. „Verbindung hergestellt“ oder Fehlermeldungen) wahlweise über das USB-Kabel oder das Netzwerk an den PC. Kann mit
- **api**: Die native Home-Assistant-API. Falls man nicht (nur) auf MQTT setzt, sorgt dieser Block für eine ultraschnelle, verschlüsselte Direktverbindung zu *Home Assistant*.
- **time**: Synchronisiert die Uhrzeit (z. B. via SNTP aus dem Internet/Schulnetzwerk). Wichtig, wenn Sensordaten direkt auf dem Gerät mit einem korrekten Zeitstempel versehen werden sollen.
- **text_sensor**: Liefert Zeichenketten statt Zahlen. Ideal, um die aktuelle WLAN-Stärke (als Text), die Uptime oder die IP-Adresse des AtomS3R im System bereitzustellen.

5. Parameter zur Laufzeit verändern

Damit der Controller nicht für jede kleine Anpassung neu geflasht werden muss:

- **number**: Erzeugt einen Schieberegler im Dashboard. Damit lässt sich ein Wert (z.B. „Kritische Temperatur, ab der ein Alarm ausgelöst wird“) zur Laufzeit live anpassen.
- **select**: Erstellt ein Dropdown-Menü (z. B. zur Auswahl von Betriebsmodi wie „Automatik“, „Manuell“, „Aus“).